

跨平台编译运行手册

前置要求

工具	版本要求	说明
CMake	>= 3.14	构建系统
C++ 编译器	支持 C++17	GCC 7+ / AppleClang 12+ / MSVC 2019+
Git	任意	仅克隆依赖源码时需要

目录结构概览

注意：C++ 源码已从仓库移除，备份为加密 zip 包（见下方）。`third_party/` 目录也已打包为 `third_party.zip`。编译前需先解压。

signal_processing_system/ ├─ config.json ├─ CMakeLists.txt ├─ build_all.bat ├─ docs/ ├─ third_party.zip ├─ cmake/ ├─ build/ ├─ output/ ├─ 01_waveform_generation.zip ├─ 02_jamming_generation.zip ├─ 03_jamming_detection_suppression.zip ├─ 04_signal_processing.zip ├─ 01_GUI_waveform/ ├─ 02_GUI_jamming/ ├─ 03_GUI_detection/ └─ 04_GUI_signal_processing/		# 外部参数配置文件(所有模块共享) # C++ 顶层构建配置 # 一键构建全部 4 个 GUI exe # 统一文档目录 # 第三方库备份 (Eigen + FFTW + mini JSON) # CMake 模块 (FindLibraries.cmake) # C++ 构建输出目录 # 运行输出目录(运行程序后生成) # 模块1加密备份 (密码: xidian_LIJINGHENG) # 模块2加密备份 # 模块3加密备份 # 模块4加密备份 # PySide6 GUI (波形生成) # PySide6 GUI (干扰生成) # PySide6 GUI (干扰识别与抑制) # PySide6 GUI (信号处理)
--	--	---

解压 zip 后将生成对应的 C++ 模块文件夹（`01_waveform_generation/` 等）和 `third_party/` 目录（含 Eigen、FFTW、mini JSON），编译所需文件齐全。

第一步：准备第三方库

自实现组件（已包含，无需额外操作）

组件	文件	替代对象	说明
mini JSON 解析器	<code>third_party/nlohmann/json.hpp</code> (~215行)	nlohmann/json (24765行)	仅实现本项目 Config.h 所需的 API 子集
Bessel 函数 I ₀	各模块 <code>Module0.h</code> 中的 <code>bessel_i0()</code>	Boost <code>cyl_bessel_i</code>	Taylor 级数展开，精度到机器 epsilon

解压 `third_party.zip`: 第三方库已打包为 `third_party.zip`, 编译前需先解压:

```
unzip third_party.zip
# 或
python -c "import zipfile;
zipfile.ZipFile('third_party.zip').extractall('.')"

```

Eigen 3.4.0 (header-only)

Eigen 是纯头文件库, 解压后在 `third_party/eigen/` 中, 无需编译, 所有平台直接可用。

FFTW 3.3.10

FFTW 是平台相关的, 必须在目标平台上重新编译。解压 `third_party.zip` 后源码在 `third_party/fftw-3.3.10/`。

macOS (AppleClang)

```
cd signal_processing_system/third_party

# 源码已包含在 third_party.zip 中, 无需下载
# 编译安装到本地 fftw-install/
rm -rf fftw-install
mkdir -p fftw-install
cd fftw-3.3.10
./configure --prefix=$PWD/../fftw-install --disable-shared --enable-static --
with-sse2 --enable-avx
make -j$(sysctl -n hw.ncpu)
make install
cd ..

```

Linux (GCC)

```
cd signal_processing_system/third_party

# 源码已包含在 third_party.zip 中, 无需下载

rm -rf fftw-install
mkdir -p fftw-install
cd fftw-3.3.10
./configure --prefix=$PWD/../fftw-install --disable-shared --enable-static --
with-sse2 --enable-avx --enable-openmp
make -j$(nproc)
make install
cd ..

```

Linux 下建议开启 `--enable-openmp`, 03模块支持OpenMP并行加速。

Windows (MSVC)

```
# 方式1: 使用vcpkg
vcpkg install fftw3:x64-windows-static

# 方式2: 手动编译(在VS Developer Command Prompt中)
# 先解压 third_party.zip
cd third_party\fftw-3.3.10
nmake /f Makefile.win32 lib
```

第三方库文件清单(编译后应存在的文件)

```
third_party/
├─ nlohmann/json.hpp          # 自实现迷你JSON解析器 (已包含)
├─ eigen/Eigen/              # Eigen头文件 (已包含)
├─ fftw-install/
│   ├─ include/fftw3.h        # FFTW头文件
│   └─ lib/libfftw3.a         # FFTW静态库 (macOS/Linux)
│       └─ lib/fftw3.lib      # FFTW静态库 (Windows)
```

第二步：编译项目

重要提示： C++ 源码已从仓库移除，需先解压对应加密 zip 包才能编译。
解压密码：xidian_LIJINGHENG

```
# 解压所有 C++ 模块（在项目根目录执行）
7z x -pxidian_LIJINGHENG 01_waveform_generation.zip
7z x -pxidian_LIJINGHENG 02_jamming_generation.zip
7z x -pxidian_LIJINGHENG 03_jamming_detection_suppression.zip
7z x -pxidian_LIJINGHENG 04_signal_processing.zip
```

解压后将生成 01_waveform_generation/、02_jamming_generation/、03_jamming_detection_suppression/、04_signal_processing/ 四个文件夹，之后按下步骤编译。

方式一：各模块独立构建（推荐）

每个模块在自己的 build/ 目录中独立构建：

```
cd signal_processing_system

# 模块1: 波形生成（同时构建 libwaveform_core.a 静态库）
cd 01_waveform_generation
mkdir -p build && cd build
cmake .. && cmake --build . -j$(sysctl -n hw.ncpu)
cd ../..

# 模块2: 干扰生成
cd 02_jamming_generation
mkdir -p build && cd build
cmake .. && cmake --build . -j$(sysctl -n hw.ncpu)
cd ../..

# 模块3: 干扰识别与抑制
cd 03_jamming_detection_suppression
```

```
mkdir -p build && cd build
cmake .. && cmake --build . -j$(sysctl -n hw.ncpu)
cd ../..

# 模块4：信号处理
cd 04_signal_processing
mkdir -p build && cd build
cmake .. && cmake --build . -j$(sysctl -n hw.ncpu)
cd ../..
```

方式二：统一构建（从项目根目录）

```
cd signal_processing_system
mkdir -p build && cd build
cmake ..
cmake --build . -j$(sysctl -n hw.ncpu)      # macOS
# cmake --build . -j$(nproc)                 # Linux
# cmake --build . --config Release           # Windows
```

编译成功后，生成4个可执行文件：

可执行文件	对应模块
01_waveform_generation/waveform_gen	波形生成
02_jamming_generation/jamming_gen	干扰生成
03_jamming_detection_suppression/jamming_det_sup	干扰识别与抑制
04_signal_processing/signal_proc	信号处理

模块01额外生成 **libwaveform_core.a** 静态库，供 01_GUI_waveform 和 04_GUI_signal_processing 的 pybind11 绑定使用。

模块02额外生成 **libjamming_core.a** 静态库，供 02_GUI_jamming 的 pybind11 绑定使用。

模块03额外生成 **libdetection_core.a** 静态库，供 03_GUI_detection 的 pybind11 绑定使用。

模块04额外生成 **libsignal_processing_core.a** 静态库，供 04_GUI_signal_processing 的 pybind11 绑定使用。

单独编译某个模块

```
# 独立构建模式
cd 01_waveform_generation && mkdir -p build && cd build && cmake .. && cmake --build . -j$(sysctl -n hw.ncpu)

# 统一构建模式下指定目标
cd signal_processing_system/build
cmake --build . --target waveform_gen
cmake --build . --target jamming_det_sup
```

库检测优先级

以下 **cmake/FindLibraries.cmake** 位于 C++ 模块 zip 包内，解压后存在于各模块目录中。

cmake/FindLibraries.cmake 按以下顺序查找库：

1. **third_party/xxx-install/** (本地编译，**推荐**)
2. **/opt/homebrew/** (macOS Homebrew)

3. `/usr/local/` (Linux 系统安装)

4. `/usr/` (Linux 系统安装)

CMake 配置时会打印检测到的库路径，确认输出中 `Eigen`、`FFTW` 路径正确。

第三步：配置参数(可选)

项目根目录的 `config.json` 包含所有可配置参数及详细注释。支持以下特性：

- **行注释**: `//` 或 `#` 开头的行会被自动忽略
- **默认值**: 删除或注释掉某行即恢复该参数的代码内置默认值
- **null值**: 设为 `null` 等同于未设置，使用默认值
- **热生效**: 修改后保存文件，重新运行程序即可，无需重新编译

智能寻址顺序

1. 环境变量 `SPS_CONFIG` 指定的路径
2. 可执行文件同目录: `./config.json`
3. 项目根目录: `../config.json`
4. 用户配置目录: `~/.config/sps/config.json`
5. 以上均未找到 → 使用全部默认值，控制台输出提示

指定自定义配置文件

```
# 方式1: 环境变量
export SPS_CONFIG=/path/to/my_config.json
./build/01_waveform_generation/waveform_gen

# 方式2: 复制到默认搜索路径
cp my_config.json ./build/config.json
cd build && ./01_waveform_generation/waveform_gen
```

第四步：运行程序

注意：以下运行命令需要先完成第二步编译（即已解压 zip 并成功构建 C++ 模块）。

所有模块自动遍历全部模式，无需人工输入。运行后输出文件统一存放到 `output/` 目录。

macOS / Linux

```
cd signal_processing_system

# 运行全部模块(按顺序)
./build/01_waveform_generation/waveform_gen
./build/02_jamming_generation/jamming_gen
./build/03_jamming_detection_suppression/jamming_det_sup
./build/04_signal_processing/signal_proc
```

Windows

```
cd signal_processing_system
build\01_waveform_generation\Release\waveform_gen.exe
build\02_jamming_generation\Release\jamming_gen.exe
build\03_jamming_detection_suppression\Release\jamming_det_sup.exe
build\04_signal_processing\Release\signal_proc.exe
```

各模块运行说明

模块	运行时间(参考)	遍历模式	输出文件
01	~1秒	5种波形	01_waveform_Case{1-5}_.dat
02	~2秒	10种干扰	02_jamming_Case{1-10}_.dat
03	~30秒	5类干扰×100CPI	03_detection_*.txt, 03_detection_*.log
04	~5秒	6种处理	04_processing_Case{1-6}_.dat

第五步：查看结果

03模块测试汇总

```
cat output/03_detection_识别与抑制日志_log.txt
```

输出示例：

```
===== 干扰识别与抑制测试 =====
参数： B=80MHz, Tp=12us, CpiNum=100

RealLabel=1 | 识别=1 (ISDJ/ISRJ/ISCJ) | 正确=100/100 | JSR干扰抑制比=18.52 dB | 耗时=0.312s
...
===== 汇总 =====
总正确： 500/500 | 总耗时： 1.523s
=====
```

输出文件格式

- .dat** 文件首行为 **行数 列数**，后续每行两个数分别表示复数的实部和虚部
- .txt** 文件每行8列：**回波实部 回波虚部 干扰实部 干扰虚部 分离干扰实部 分离干扰虚部 分离目标实部 分离目标虚部**

常见问题

Q: CMake报错找不到Eigen

确保已解压 `third_party.zip`，且 `third_party/eigen/Eigen/Dense` 文件存在。如使用系统安装的Eigen，检查路径：

```
ls /opt/homebrew/include/eigen3/Eigen/Dense # macOS
ls /usr/include/eigen3/Eigen/Dense          # Linux
```

Q: CMake报错找不到FFTW

确保已解压 `third_party.zip` 并完成 FFTW 编译（见第一步），且 `third_party/fftw-install/lib/libfftw3.a` 存在。

Q: macOS下OpenMP不可用

AppleClang不支持OpenMP，03模块会自动回退到单线程模式，功能完全正常。Linux下GCC默认支持OpenMP。

Q: 04模块Case6输出全零或nan

当输入信号 `Radar_Sig` 全零时(默认初始化)，`JamTarDivi` 产生nan输出，属于正常现象，不影响功能。

Q: [Config] 未找到外部配置文件

这是正常提示。程序会使用代码内置默认值运行。如需使用自定义参数，将 `config.json` 放到项目根目录或设置 `SPS_CONFIG` 环境变量。

Q: IDE的LSP报大量错误(Eigen/Config找不到等)

这是因为IDE没有cmake的include路径，不影响实际编译。`cmake --build` 可以正常编译通过。

Q: 重新编译

```
# 独立构建模式
cd 01_waveform_generation && rm -rf build && mkdir build && cd build && cmake
.. && cmake --build . -j$(sysctl -n hw.ncpu)

# 统一构建模式
cd signal_processing_system && rm -rf build && mkdir build && cd build && cmake
.. && cmake --build . -j$(sysctl -n hw.ncpu)
```

完整一键脚本 (macOS/Linux)

注意：运行此脚本前，需先解压 4 个 C++ 模块的加密 zip 包（密码：`xidian_LIJINGHENG`），使 `01_waveform_generation/`、`02_jamming_generation/`、`03_jamming_detection_suppression/`、`04_signal_processing/` 目录存在。

```
#!/bin/bash
set -e
```

```

PROJECT_DIR="$(cd "$(dirname "$0")" && pwd)"
cd "$PROJECT_DIR"

# 检查 C++ 模块目录是否存在, 不存在则提示解压 zip
for mod in 01_waveform_generation 02_jamming_generation
03_jamming_detection_suppression 04_signal_processing; do
    if [ ! -d "$PROJECT_DIR/$mod" ]; then
        echo "错误: 目录 $mod/ 不存在, 请先解压对应的加密 zip 包(密码:
xidian_LIJINGHENG)"
        exit 1
    fi
done

if [ ! -d "$PROJECT_DIR/third_party" ]; then
    echo "错误: third_party/ 目录不存在, 请先解压 third_party.zip"
    exit 1
fi

echo "=== 1. 编译FFTW ==="
cd third_party
if [ ! -f fftw-3.3.10/configure ]; then
    curl -O https://www.fftw.org/fftw-3.3.10.tar.gz
    tar xzf fftw-3.3.10.tar.gz
fi
cd fftw-3.3.10
./configure --prefix=$PWD/../fftw-install --disable-shared --enable-static
make -j$(nproc 2>/dev/null || sysctl -n hw.ncpu)
make install

echo "=== 2. 编译项目 ==="
cd "$PROJECT_DIR"

# 各模块独立构建
for mod in 01_waveform_generation 02_jamming_generation
03_jamming_detection_suppression 04_signal_processing; do
    echo "--- 构建 $mod ---"
    cd "$PROJECT_DIR/$mod"
    rm -rf build
    mkdir build && cd build
    cmake .. && cmake --build . -j$(nproc 2>/dev/null || sysctl -n hw.ncpu)
done

echo "=== 3. 运行测试 ==="
cd "$PROJECT_DIR"
for mod in 01_waveform_generation 02_jamming_generation
03_jamming_detection_suppression 04_signal_processing; do
    echo "--- 运行 $mod ---"
    ./$mod/build/${basename $mod | sed 's/^[0-9]*_//' | sed 's/_.*//'}_gen
2>/dev/null || \
    ./$mod/build/${cat $mod/CMakeLists.txt | grep -m1 'add_executable' | sed
's/.*([^\ ]*).*/\1'}
done

echo "=== 完成! 输出文件在 output/ 目录 ==="

```

使用方式: 保存为 `build_and_run.sh`, 放到项目根目录, 执行 `chmod +x build_and_run.sh && ./build_and_run.sh`。

第六步：GUI 应用（可选）

项目提供四个独立的 PySide6 GUI 应用，分别封装模块01~04。

各 GUI 概览

GUI	封装模块	C++ 绑定	静态库依赖
01_GUI_waveform	模块01 (5种波形)	<code>waveform_bind.cpp</code> → <code>waveform_cpp.pyd</code>	<code>libwaveform_core.a</code> (Eigen)
02_GUI_jamming	模块02 (10种干扰)	<code>jamming_bind.cpp</code> → <code>jamming_cpp.pyd</code>	<code>libjamming_core.a</code> (Eigen + FFTW3)
03_GUI_detection	模块03 (5种干扰类型)	<code>detection_bind.cpp</code> → <code>detection_cpp.pyd</code>	<code>libdetection_core.a</code> (Eigen + FFTW3)
04_GUI_signal_processing	模块 04+01 (6 种处理)	<code>signal_processing_bind.cpp</code> → <code>signal_processing_cpp.pyd</code>	<code>libsignal_processing_core.a</code> + <code>libwaveform_core.a</code> (Eigen + FFTW3)

四个 GUI 通过 pybind11 在内存中传递 numpy 数组，不经过文件 I/O。

运行方式（开发模式）

```
cd 01_GUI_waveform && pip install -r requirements.txt && python app.py
cd 02_GUI_jamming && pip install -r requirements.txt && python app.py
cd 03_GUI_detection && pip install -r requirements.txt && python app.py
cd 04_GUI_signal_processing && pip install -r requirements.txt && python app.py
```

Windows 一键打包为 exe

注意：打包前需先解压 4 个 C++ 模块的加密 zip 包（密码：`xidian_LIJINGHENG`）和 `third_party.zip`，使 C++ 源码目录和第三方库目录存在。

```
# 先构建 4 个 C++ 静态库
cd 01_waveform_generation && mkdir -p build && cd build && cmake .. && cmake --build .
cd ../../02_jamming_generation && mkdir -p build && cd build && cmake .. && cmake --build .
cd ../../03_jamming_detection_suppression && mkdir -p build && cd build && cmake .. && cmake --build .
cd ../../04_signal_processing && mkdir -p build && cd build && cmake .. && cmake --build .

# 一键打包全部 4 个 GUI
cd ../../
build_all.bat
```

各 GUI 也可单独打包：

```
cd 01_GUI_waveform && scripts\build.bat
cd 02_GUI_jamming && scripts\build.bat
cd 03_GUI_detection && scripts\build.bat
cd 04_GUI_signal_processing && scripts\build.bat
```

打包产物位于各 **dist/** 目录下的单文件 exe（如 **dist/雷达波形生成系统.exe**），可直接拷贝到任意 Windows 机器运行。
config.json 需放在 exe 上两级目录（项目根目录），或通过菜单 **文件** → **加载配置...** 手动导入。

C++ 源码分发

项目根目录提供四个 C++ 模块的加密 zip 包（已排除 build/output/venv 等构建产物）：

文件	模块
01_waveform_generation.zip	波形生成
02_jamming_generation.zip	干扰生成
03_jamming_detection_suppression.zip	干扰识别与抑制
04_signal_processing.zip	信号处理

解压密码：**xidian_LIJINGHENG**

GUI 构建前置条件

- **third_party.zip** 已解压（**third_party/** 目录存在，含 Eigen + FFTW）
- 对应模块的 C++ 源码已解压（解压加密 zip 包，密码：**xidian_LIJINGHENG**）
- 对应模块的 C++ 静态库已构建完成
- Python 3.13+
- pybind11（编译时需要）
- PySide6, PyQtgraph, numpy, scipy（运行时）
- PyInstaller（打包时，可选）

GUI 可视化功能

GUI	图表标签页	特色功能
01_GUI_waveform	时域/频谱/信号矩阵/相位谱/载频序列/随机相位/STFT/ 距离-多普勒	RD 物理坐标 (m/m/s)
02_GUI_jamming	时域/频域/信号矩阵/ 距离-多普勒 /STFT/脉冲对比/拖引轨迹	RD 物理坐标 (m/m/s)
03_GUI_detection	时域/频域/信号矩阵/距离像/目标分离/ 距离-多普勒 /包络	载波去除 + RD 图 (Ka波段)
04_GUI_signal_processing	脉冲特性/频域/信号矩阵/ 距离-多普勒 /解耦结果/ STFT时频 /结果汇总	passband 载波去除 STFT

详细文档：

- **C++ 模块详解**
- **GUI 使用指南**
- **GUI 共享组件**